# Research Trail Builder: A Graph-Based Knowledge Map for Scientific Literature Navigation

Howard Liu
University of Illinois at Urbana-Champaign
USA
yl140@illinois.edu

Lawrence Wang
University of Illinois at Urbana-Champaign
USA
lw41@illinois.edu

Eric Chen
University of Illinois at Urbana-Champaign
USA
ericzc2@illinois.edu

Haoyang Wang
University of Illinois at Urbana-Champaign
USA
hw86@illinois.edu

## Abstract

Research Trail Builder is a web application that transforms a textual research topic query into an interactive, graph-based knowledge map of the scientific literature. Given a query, the system decomposes it into sub-problems, retrieves relevant papers from three academic databases (OpenAlex, Semantic Scholar, and arXiv), extracts structured claims and methods from each paper using an LLM, assembles a navigable concept graph, synthesizes a written literature summary, and surfaces research gaps. The entire pipeline is orchestrated with LangGraph and displayed through a Streamlit web interface. For example, on a representative query about transformer models for protein folding, the system retrieved 20–22 papers, extracted 92–103 distinct claims and 57–63 methods, and produced a concept graph with up to 286 directed edges across three pipeline runs. Source code is available at https://github.com/HowardLiu0830/CS510-final-project.

## Keywords

Literature Navigation, Concept Graph, LangGraph, LLM Extraction, Academic Search, Knowledge Map

## 1 Introduction

The volume of published scientific literature has grown to a point where manual literature review is a significant bottleneck for researchers. Tools such as Google Scholar and Semantic Scholar return ranked lists of papers, but do not communicate how ideas relate to each other, evolve, or conflict. A student or researcher entering a new subfield may spend weeks constructing a mental map before they can meaningfully contribute.

Research Trail Builder directly addresses this pain point. Rather than returning a flat list of results, the system constructs an interactive *concept graph*: a visual representation of the key claims, methods, and relationships across retrieved papers. Shared concepts from different papers merge into a single node, surfacing cross-paper connections that would otherwise require manual synthesis. Research gaps are explicitly labeled so that users can identify underexplored directions.

The primary users are graduate students, researchers, and practitioners in academia and industry who need to quickly understand the landscape of a research area without reading dozens of papers individually. In particular, our system lowers the entry barrier for junior researchers, and provides experienced researchers with a rapid overview when entering adjacent fields.

## 2 Related Work

Connected Papers [2] and Litmaps [3] visualize citation networks, showing which papers cite which. Research Trail Builder differs by constructing a *semantic* concept graph from LLM-extracted claims and methods rather than from citation links, which surfaces thematic connections between papers that do not directly cite each other.

Elicit [4] uses LLMs to answer research questions from the literature, but presents results as a structured table rather than a graph. Our system emphasizes the visual and navigable aspect of concept relationships.

Retrieval-augmented generation (RAG) systems [5] retrieve relevant documents to answer questions, but do not construct an explicit graph of conceptual relationships. The LangGraph orchestration in Research Trail Builder is more similar to agent-based RAG pipelines [6], but specialized for the literature-survey task with a domain-specific graph output.

## 3 System Overview

### 3.1 Architecture

Figure 1 illustrates the high-level architecture. The system consists of four layers: a *search layer* that queries academic databases, an *agent layer* that orchestrates the pipeline with LangGraph, a *graph layer* that builds and enriches the concept graph, and a *presentation layer* that renders the interactive Streamlit UI.

At a high level, the system follows an end-to-end query-to-graph workflow. A user first enters a free-text research topic through the Streamlit interface. The agent layer decomposes the topic into several focused sub-problems, which are then sent to the search layer to retrieve papers from OpenAlex, Semantic Scholar, and arXiv. Retrieved papers are deduplicated and cached before being passed to the extraction stage, where an LLM extracts structured claims, methods, evidence, and potential gaps. The graph layer converts these structured outputs into paper and concept nodes with typed edges, and the presentation layer renders the result as an interactive concept graph together with a narrative synthesis and expandable paper details.

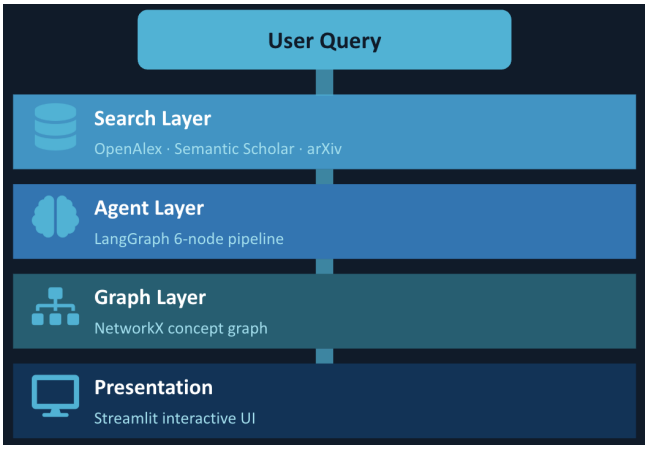


**Figure 1: High-level system architecture.**

### 3.2 Technology Stack

- **Orchestration**: LangGraph
- **LLM**: OpenAI GPT models via structured output
- **Academic search**: OpenAlex, Semantic Scholar, arXiv APIs
- **Graph construction**: NetworkX; serialized for streamlit-agraph (vis.js)
- **Web interface**: Streamlit
- **Language / packaging**: Python 3.11, conda or Python venv

## 4 Implementation

### 4.1 LangGraph Pipeline

The pipeline is a directed acyclic graph of six LangGraph nodes, compiled from a `StateGraph[ResearchState]`:

`scope_query` → `search_papers` → `screen_and_extract` → `build_graph` → `synthesize` → `identify_gaps`

Each node receives the full pipeline state and returns a partial update dict. The shared state carries the query, decomposed sub-problems, retrieved papers, per-paper extractions, the concept graph, the synthesized summary, and identified gaps.

*scope_query.* The first node uses an LLM with structured output to decompose the user's query into 3–5 concrete sub-problems. These sub-problems guide subsequent search and ensure diverse coverage. For example, the query "transformer models for protein folding" is decomposed into sub-problems such as "transformer architectures for protein structure prediction," "geometric/equivariant transformers for 3D protein folding," and "datasets and benchmarks for transformer-based protein folding models."

*search_papers.* Papers are retrieved by issuing parallel searches for the main query and each sub-problem across all three academic search sources (up to six total queries, capped to prevent excessive API calls). Results are deduplicated by DOI or normalized title across sources. A disk cache avoids redundant API calls on repeated or overlapping sub-problems.

**Listing 1: Pseudocode for concept graph construction.**

```
for each extraction E:
  add paper node for E.paper_id
  for each claim C in E.claims:
    nid = "claim:" + normalize(C)
    add node nid if new (label=C)
    add edge (paper, nid, relation="asserts")
  for each method M in E.methods:
    nid = "method:" + normalize(M)
    add node nid if new (label=M)
    add edge (paper, nid, relation="uses")

for each paper P:
  for each pair (M1, M2) of methods used by P:
    rel = token_overlap_relation(M1, M2)
    add edge (M1, M2, relation=rel)
```

*screen_and_extract.* Each retrieved paper is passed to an LLM extractor that produces up to five claims, three methods, and three pieces of evidence, along with a confidence score. Extractions run concurrently (configurable parallelism) because each call is dominated by a network round-trip; serial execution on 20 papers is the dominant bottleneck of the full pipeline.

*build_graph.* The graph builder converts extractions into a NetworkX directed graph. Each paper becomes a node. Each unique claim and method becomes a concept node; concept IDs are normalized (lowercase, collapsed whitespace) so that the same concept mentioned by multiple papers merges into one node. Edges from papers to claims carry the label `asserts`; edges to methods carry `uses`. Methods from the same paper are also connected by `Builds Upon` or `Contrasts` edges, inferred from token overlap.

*synthesize.* An LLM writes a 3–5 paragraph narrative synthesis of the retrieved literature, citing paper titles inline, structured around the sub-problems.

*identify_gaps.* A second LLM call uses the synthesis and graph statistics (node counts, top cross-referenced concepts, niche concepts cited by only one paper) to surface 3–5 research gaps.

### 4.2 Concept Graph Construction

Algorithm 1 summarizes the graph-building logic.

### 4.3 Evaluation Framework

The system includes a built-in evaluation framework using an LLM as judge, scoring outputs on four dimensions on a 1–5 scale: *relevance*, *coverage*, *structural organization*, and *insightfulness*. A human evaluation scaffold generates structured forms for domain-expert reviewers. The command-line evaluation workflow is:

```
research-trail-agent "your query"
research-trail-runs-to-jsonl \
  --output data/eval/results.jsonl
research-trail-eval \
  --input data/eval/results.jsonl \
  --output data/eval/scores.json
```

# 5 Using the Software

## 5.1 Installation

```
# Clone and create environment
git clone https://github.com/HowardLiu0830/\
  CS510-final-project
conda env create -f environment.yml
conda activate cs510
pip install -e .

# Configure API key
cp .env.example .env
# Set OPENAI_API_KEY in .env

# Launch the web app
streamlit run app/streamlit_app.py
```

For users without conda, the project can also be installed with Python venv:

```
python3.11 -m venv .venv
source .venv/bin/activate

pip install -r requirements.txt
pip install -e .

cp .env.example .env
# Set OPENAI_API_KEY in .env

streamlit run app/streamlit_app.py
```

## 5.2 Web Interface Walkthrough

*Step 1 — Enter a query.* On the main page, type a research topic into the search box (e.g., "graph neural networks for drug discovery") and click **Run** as shown in Figure 2.

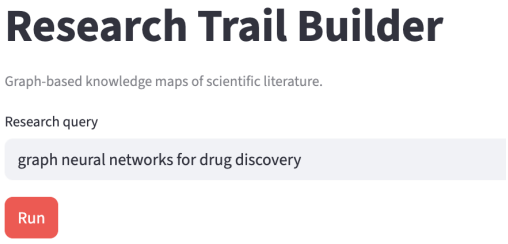


**Figure 2: Query entry interface.**

*Step 2 — View the concept graph.* After the pipeline completes (typically 1–3 minutes for a live query), the app renders an interactive force-directed graph. Blue circles labeled P1, P2, etc. represent papers; green circles represent extracted claims; orange circles represent methods. Single-clicking a node displays its label; double-clicking a paper node opens its source URL in a new tab. Edge labels (`asserts`, `uses`) are shown on hover. An illustration of the graph is shown in Figure 3.

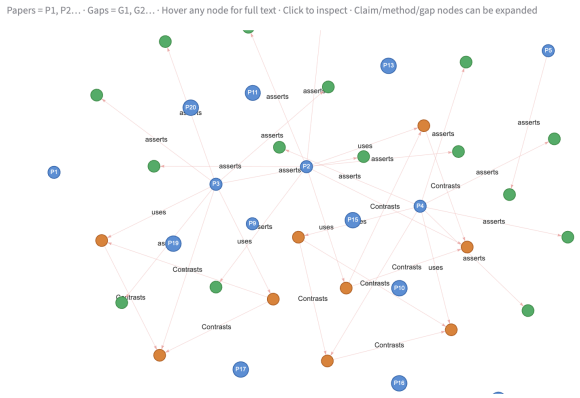


**Figure 3: Interactive concept graph.**

*Step 3 — Read the synthesis.* Below the graph, a written narrative synthesis summarizes the main subtopics and findings, with paper titles cited inline.

*Step 4 — Explore gaps and papers.* Expandable sections list the identified research gaps (G1, G2, …), the full paper index with abstracts, and per-paper claim/method details.

*Step 5 — Expand the trail.* The **Expand Topic** button appends additional papers and concepts to the existing graph by issuing a new search on a user-specified follow-up query, enabling iterative drill-down without starting over. This is displayed under the node details as shown in Figure 4.

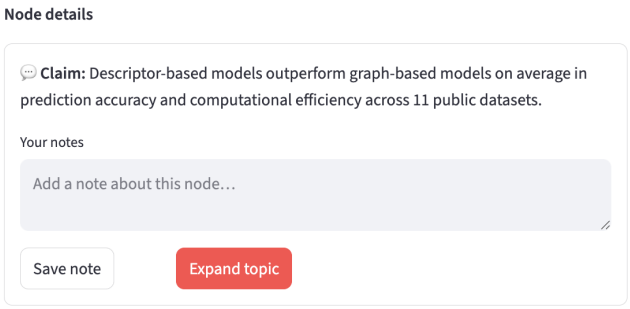


**Figure 4: Expand topic and paper index panel.**

## 5.3 Command-Line Interface

A CLI is also available for batch use:

```
# Run one query and save artifacts
research-trail-agent \
  "retrieval-augmented generation for QA"

# View saved run
ls data/runs/
cat data/runs/<timestamp>__<slug>/state.json
```

## 6 Evaluation

Because Research Trail Builder is an applied tool that produces open-ended literature surveys, there is no single ground-truth label to score against. Our evaluation therefore combines three complementary views, all run over the same fixed 20-query benchmark. First, we report *functional* statistics (what the pipeline actually produces: retrieved papers, concept nodes, multi-cite concepts, and edges) to characterize the output without any quality judgement. Second, we score every run with an *LLM-as-judge* rubric on eight dimensions covering both the written literature review (relevance, coverage, groundedness, synthesis) and the identified research gaps (relevance, specificity, novelty, significance), each on a 1–5 scale. Third, we run three targeted comparisons to attribute the resulting quality to the specific design choices in our pipeline: a *baseline comparison* against a zero-shot LLM and a retrieval-only system to isolate the contribution of the concept graph, a *model choice* comparison across three candidate generation LLMs to justify the default model, and a *number-of-sub-queries* comparison to justify how aggressively the system decomposes the user's query. The remainder of this section walks through each of these in turn.

### 6.1 Test Query Set

All evaluation experiments use the same fixed set of **20 queries** designed to span six current research areas in ML/AI. Category counts are deliberately balanced (2–5 queries per bucket) so that no single subfield dominates the aggregate scores. The full distribution is shown in Table 1 and visualized in Figure 5.

**Table 1: Test query set: 20 queries across six research categories.**

| Category | Count | Example query |
|---|---|---|
| LLM alignment / eval | 4 | RLHF alignment |
| ML methodology | 5 | NAS efficiency |
| ML × Bio/Chem | 3 | GNN for drug discovery |
| Vision / Multimodal | 3 | Continual learning for ViTs |
| LLM architecture | 3 | Mixture-of-experts routing |
| LLM applications | 2 | RAG for scientific QA |
| **Total** | **20** | |

The same query set is reused across all ablations (generation-model, system-configuration, and sub-query-count), so differences between conditions reflect changes in the system rather than changes in the evaluation workload.

### 6.2 Baseline Comparison

To isolate the contribution of each component of the pipeline, we compare the full system against two ablated baselines on the same 20-query set, holding the generation model fixed at `gpt-5-mini` and using a single LLM judge (`deepseek-v4-flash`). All three conditions are scored on the same eight-dimension rubric (four *review* dimensions and four *gap* dimensions, 1–5 each).

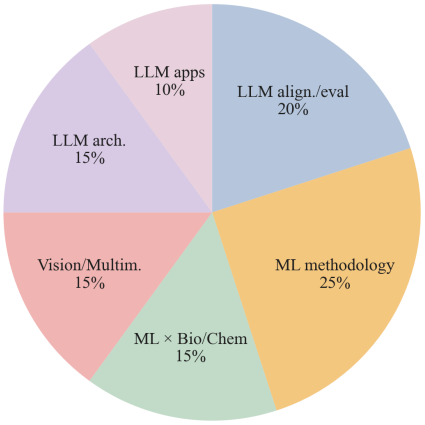


**Figure 5: Distribution of the 20 evaluation queries across six research categories.**

- **zero_shot**: the LLM answers the query from memory, with no retrieval and no graph. Tests how much of the literature-review task can be done by recall alone.
- **no_graph**: retrieval is enabled and the LLM is prompted with retrieved abstracts, but the concept graph is not built and no graph statistics are passed to the gap finder.
- **Our**: the complete pipeline, including concept-graph construction and graph-statistic features for gap identification.

**Table 2: Baseline comparison on 20 queries (gpt-5-mini, single judge). Bold marks the best score per row.**

| Dimension | zero_shot | no_graph | Ours |
|---|---|---|---|
| review_relevance | **5.00** | 4.95 | **5.00** |
| review_coverage | 4.40 | 4.10 | **4.50** |
| review_groundedness | 1.15 | 4.20 | **4.75** |
| review_synthesis | 4.70 | 4.75 | **5.00** |
| gap_relevance | 1.00 | **5.00** | **5.00** |
| gap_specificity | 1.00 | 4.55 | **4.70** |
| gap_novelty | 1.00 | 3.95 | **4.00** |
| gap_significance | 1.00 | **4.90** | **4.90** |
| review_mean | 3.81 | 4.50 | **4.81** |
| gap_mean | 1.00 | 4.60 | **4.65** |
| **overall** | 2.41 | 4.55 | **4.73** |

*Retrieval is the dominant lift.* The largest jump is `zero_shot` → `no_graph` (2.41 → 4.58 overall). Without retrieval, the LLM scores 1.15 on groundedness, indicating that paper titles and findings are fabricated; adding retrieval moves groundedness to 4.20. The gap rubric goes from 1.0 to 4.65 because `zero_shot` produces an empty gap list by design.

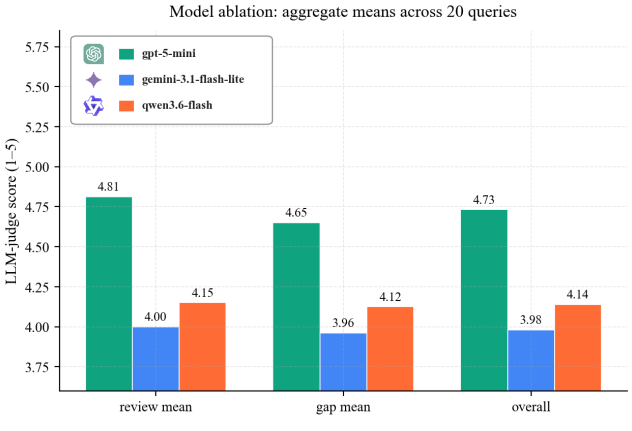


**Figure 6: Aggregate LLM-judge scores (review mean, gap mean, overall; 1–5) for the three candidate generation models on the 20-query evaluation set.**

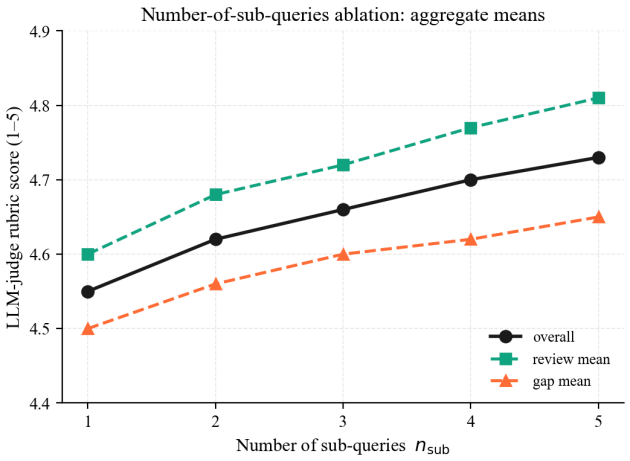


**Figure 7: LLM-judge aggregate scores as a function of the number of sub-queries $n_{sub}$, on the 20-query evaluation set.**

*The concept graph adds a modest but real improvement.* On top of retrieval, enabling the full concept graph improves the overall score from 4.58 to 4.71 (+0.13), with the largest gain on `review_groundedness` (+0.55) and `review_synthesis` (+0.25). The gap-side dimensions are essentially tied between `no_graph` and `full`, suggesting that the current graph features help the synthesis step more than the gap-finding step.

*The literature-review prose is largely model-driven.* `zero_shot` still scores 4.70 on `review_synthesis` and 4.40 on `review_coverage`, indicating that prose quality and topical breadth come mostly from the underlying LLM. The system's measurable value is concentrated in *groundedness* (verifiable citation to retrieved papers) and *gap identification*, not in the written narrative itself.

## 6.3 Model Choice

Because Research Trail Builder is an applied tool that end users run on their own queries, the choice of generation LLM is a deployment decision rather than just a research variable. We considered three candidate models that are widely available, cost-friendly, and supported by our orchestration stack: `gpt-5-mini` (OpenAI), `google/gemini-3.1-flash-lite`, and `qwen/qwen3.6-flash` (both via OpenRouter). To pick between them we ran the full pipeline end-to-end on all 20 evaluation queries under each model, holding the retrieval layer, prompts, judge, and graph-construction code constant. Figure 6 compares the resulting aggregate rubric scores.

*Why we picked gpt-5-mini.* `gpt-5-mini` produces the best output on every aggregate (review mean 4.81, gap mean 4.65, overall 4.73) and on every individual rubric dimension. The next-best candidate (`qwen3.6-flash`) trails by −0.59 overall, and `gemini-3.1-flash-lite` trails by −0.75. The gap is largest exactly where it matters most for this kind of tool: cross-paper synthesis (`review_synthesis` spread 1.25) and concrete gap identification (`gap_specificity` spread 1.05). For a user who opens the app to understand a new subfield in a few minutes, those two dimensions are what makes the output feel like a real literature review instead of a list of abstracts, so we ship the system with `gpt-5-mini` as the default.

## 6.4 Number of Sub-Queries

A second design choice that directly affects what the user sees is *how many sub-queries to decompose the input into* at the `scope_query` stage. More sub-queries give the search layer more angles to attack the topic from, but each sub-query costs an extra LLM call and an extra round of API requests across three search backends, so it directly increases latency and dollar cost. To pick a sensible default we varied $n_{sub} \in \{1, 2, 3, 4, 5\}$, held everything else constant, and ran each variant on all 20 queries. Figure 7 shows the aggregate trend.

*More sub-queries produce better output.* Going from $n_{sub} = 1$ to $n_{sub} = 5$, all three aggregate scores rise monotonically: overall 4.55 → 4.73, review mean 4.60 → 4.81, and gap mean 4.50 → 4.65. The lift is concentrated on the dimensions where single-query search tends to fall short: `review_synthesis` rises 4.65 → 5.00, `gap_significance` rises 4.70 → 4.90, and `gap_novelty` rises 3.60 → 4.00. Decomposing the topic forces the agent to surface several distinct angles and to write a synthesis that weaves them together, instead of restating one search result.

*Why we default to multiple sub-queries.* The single-query baseline already scores well (4.55 overall), but each additional sub-query reliably improves the qualitative parts of the output that users actually read (narrative synthesis and the concreteness/significance of identified gaps), with the curve peaking at $n_{sub} = 5$ (+0.18 overall vs. $n_{sub} = 1$). We therefore ship the system with $n_{sub} = 5$ as the default, matching our configuration. Users who need cheaper or faster runs can dial $n_{sub}$ down at the cost of a predictable, mostly-linear drop in synthesis and gap quality.

## 6.5 Graph Statistics

Table 3 reports the average structural output of the pipeline on the full 20-query evaluation set under the headline "Ours" configuration

(gpt-5-mini, full pipeline with concept-key clustering at threshold 0.875).

**Table 3: Average pipeline output per query, aggregated over the 20-query evaluation set (Ours configuration).**

| Quantity | Mean per query |
|---|---|
| Retrieved papers (after deduplication) | 12.7 |
| Concept nodes (claims + methods) | 79.1 |
| Multi-cite concepts ($\geq$ 2 papers) | 5.3 |
| Directed edges | 122.4 |

The 5.3 multi-cite concepts per query confirm that concept-key clustering reliably surfaces cross-paper structure, which is the central function of the graph layer; without clustering this number would collapse to zero. Variation across queries reflects non-determinism in both LLM extraction and API result ordering, as well as topic-specific differences in literature density, while niche or very recent topics yield sparser ones. Repeated runs on the same query (protein folding, $n = 3$) produced stable LLM-judge scores across all four rubric dimensions, confirming that output quality is consistent under identical conditions.

## 6.6 Functional Testing

In addition to live pipeline runs, we tested core system functions using unit and smoke tests. These tests cover package import and pipeline initialization, offline-mode execution without API calls, search aggregation and deduplication, search client contracts, and evaluation result serialization. The purpose of these tests is to ensure that the system remains reproducible in a development environment even when external APIs or LLM keys are unavailable.

**Table 4: Functional testing coverage.**

| Component | Tested Behavior | Result |
|---|---|---|
| Smoke test | Package import and pipeline initialization | Pass |
| Search aggregator | Multi-source aggregation and deduplication | Pass |
| Search contracts | Search client output format consistency | Pass |
| Offline mode | Run without OpenAI API key or network calls | Pass |
| Evaluation | JSONL conversion and rubric workflow | Pass |

## 6.7 Limitations

Extraction quality degrades on papers with short or uninformative abstracts; the extractor returns an empty extraction rather than hallucinating. The graph can become visually dense for broad queries with many papers; the top-30 concept filter in the UI mitigates this. The system currently does not support full-text extraction, relying solely on abstracts.

# 7 Case Study

To illustrate the system end-to-end, we trace a complete run on the query "transformer models for protein folding."

## 7.1 Query Decomposition

The `scope_query` node decomposes the query into four fine-grained sub-problems:

(1) Transformer architectures for protein structure prediction
(2) Geometric/equivariant transformers for 3D protein folding
(3) Datasets and benchmarks for transformer-based protein folding models
(4) Evaluation metrics and baselines for transformer-based protein structure prediction

These sub-problems guided parallel searches across OpenAlex, Semantic Scholar, and arXiv, returning 21 papers after deduplication.

## 7.2 Extraction Examples

Table 5 shows representative LLM extractions from two papers. ProtGPT2 [1] and OmegaFold each contributed two claims and two methods; these were normalized and added as concept nodes in the graph.

**Table 5: Sample extractions from two papers in the protein folding run.**

| Paper | Claims (excerpt) | Methods |
|---|---|---|
| ProtGPT2 | "Generates de novo sequences following natural protein principles"; "88% predicted globular" | Unsupervised protein LM; Disorder-based globularity prediction |
| OmegaFold | "First method to predict high-resolution structure from a single sequence"; "No MSA required" | Single-sequence protein LM; Geometry-inspired transformer |

## 7.3 Concept Graph and Synthesis

After processing all 21 papers, the graph contained 97 claim nodes, 60 method nodes, and 157 directed edges. Several method nodes merged across papers: "attention mechanism" appeared in extractions from five papers and collapsed into a single node with five incoming `uses` edges. Similarly, "multiple sequence alignment (MSA)" appeared in three papers — spanning both MSA-dependent and MSA-free approaches. The `Contrasts` edge between "MSA-based structure prediction" and "single-sequence structure prediction" was inferred automatically from disjoint token sets.

The synthesis narrative identified three main threads: (1) sequence-only design (ProtGPT2), (2) structure prediction without MSAs (OmegaFold), and (3) critical perspectives on the protein-as-language analogy. The `identify_gaps` node surfaced five gaps, including:

- *Lack of standardized benchmarks* for transformer-based protein folding that test across varying information regimes.

- *Limited experimental validation* of de novo sequences from language models, where evaluations rely primarily on AlphaFold predictions rather than wet-lab confirmation.
- *Efficiency gaps*: need for resource-constrained architectures and benchmarks for long sequences.

## 8 Design Decisions and Challenges

*Coverage versus runtime.* The default search configuration can retrieve up to six query variants across three academic search APIs. This improves topical coverage but increases runtime because each retrieved paper may trigger a separate LLM extraction call. We therefore cap the number of query variants and papers per source, and use disk caching to avoid repeated API calls during iterative development.

*Grounded extraction versus hallucination.* A free-form LLM summary can introduce unsupported claims. To reduce this risk, the system first extracts structured claims, methods, evidence, and confidence scores from retrieved papers, and then builds the graph from these structured fields instead of directly generating a graph from the user query.

*Graph readability.* Concept graphs can become visually dense for broad queries. The interface uses color-coded node types, hover/click inspection, and filtering of top concepts to make the graph easier to interpret. However, graph layout, zoom control, and large-graph navigation remain important areas for future improvement.

## 9 Team Contributions

**Lawrence Wang:** designed and implemented the LangGraph agent framework and backend pipeline, including the `scope_query`, `search_papers`, `screen_and_extract`, `build_graph`, `synthesize`, and `identify_gaps` nodes, the offline mode infrastructure, and the run-log artifact system.
**Eric Chen:** implemented the academic search layer (OpenAlex, Semantic Scholar, and arXiv clients), the search aggregator with deduplication and disk caching, the paper parsing and data model, and the Streamlit web interface including the interactive graph visualization, paper index, and expand-topic functionality.
**Haoyang Wang:** designed the LLM prompts for query decomposition, extraction, synthesis, and gap identification, and output refinement to ensure consistent JSON from the LLM across prompt variations and model versions.
**Howard Liu:** built the evaluation framework, including the LLM-as-judge pipeline with the four-dimension rubric, the human evaluation scaffold and form generator, the `runs_to_jsonl` collector, and the evaluation CLI entry points.

## 10 Conclusion

Research Trail Builder demonstrates that a six-node LangGraph pipeline backed by multi-source academic search and LLM extraction can produce a meaningful concept graph from a single free-text query in under three minutes. The resulting graph reveals cross-paper connections, highlights shared methods, and surfaces research gaps that would otherwise require hours of manual reading. Future work includes full-text extraction, incremental graph refinement as the user explores, and integration of citation-network data to complement the semantic graph.

## References

[1] N. Ferruz, S. Schmidt, and B. Höcker, "ProtGPT2 is a deep unsupervised language model for protein design," *Nature Communications*, vol. 13, no. 4348, 2022.
[2] Connected Papers, https://www.connectedpapers.com/, 2020.
[3] Litmaps, https://www.litmaps.com/, 2021.
[4] Ought, "Elicit: The AI Research Assistant," https://elicit.com/, 2022.
[5] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *NeurIPS*, 2020.
[6] N. Shinn et al., "Reflexion: Language agents with verbal reinforcement learning," *NeurIPS*, 2023.